南京邮电大学毕业设计(论文)开题报告

题 目	数组类型变量状态引导的模糊测试工具的设计与实现				
学生姓名	孙琳	班级学号	B22042210	专业	软件工程

一、课题任务学习与理解

在众多的软件测试技术中，模糊测试凭借其自动化程度高、漏洞发现能力强、实施成本低等优势，逐渐成为工业界和学术界广泛关注的研究热点。近年来，随着 AFL、LibFuzzer 等开源模糊测试工具的普及，模糊测试技术得到了快速发展和广泛应用。然而，传统的模糊测试技术主要采用基于覆盖率增长的导向策略，通过代码覆盖率、边覆盖率等指标来评估测试用例的质量，并优先选择能够增加覆盖率的种子进行变异。这种以覆盖率为单一目标的测试策略虽然能够提高代码覆盖的广度，但在触发与特定变量状态相关的错误方面表现不足。

数组作为一种基本且重要的数据结构，在各类软件程序中得到了广泛的应用。然而，数组相关的安全问题一直是软件漏洞的重要来源。根据 CWE 数据库的统计，与数组操作相关的漏洞，如 CWE-119（缓冲区错误）、CWE-125（越界读取）、CWE-787（越界写入）等，长期占据最常见软件漏洞榜单的前列。现有的模糊测试工具在处理数组相关代码时，通常将数组视为普通的内存区域进行模糊测试，缺乏对数组变量状态的精细化建模和引导，导致许多与数组状态相关的漏洞无法被有效发现。

针对上述问题，本课题提出了一种基于数组类型变量状态引导的模糊测试方法。该方法的核心思想是，在现有模糊测试工具的基础上，引入数组类型变量状态的多样化和特定状态作为引导机制，通过精细化地建模和分析数组变量的状态，设计状态多样性度量方法和基于状态的测试用例生成策略，从而提高模糊测试对数组相关错误的触发能力。

二、文献综述

模糊测试技术自 20 世纪 90 年代提出以来，经历了从盲目随机测试到定向引导测试的发展历程。随着研究的深入，通过引入特定的引导机制将测试资源集中在目标区域或目标代码上，可以显著提高测试的效率和漏洞发现能力，这一认识推动了定向模糊测试技术的快速发展。在定向引导机制的设计方面，国外学者提出了多种创新方法。基于距离的引导策略是其中重要的研究方向之一。Böhme[1]等人提出了一种基于代码距离的灰箱模糊种子排序方法，该方法通过分析测试用例与目标代码之间的代码结构距离，优先选择更有可能触发目标覆盖的测试种子进行变异和执行。虽然该方法突破了传统覆盖率度量的局限，但其引导目标仍然是代码覆盖率的增长，而非程序状态的多样性。在基于代码结构引导的基础上，Petsios[2]等人进一步将函数调用图分析与模糊测试相结合，提出了 FunFuzz 系统。该方法通过构建函数调用图和依赖关系分析，根据函数在程序中的重要性评分来分配测试资源。实验结果表明，这种方法能够识别程序中的关键安全函数并对这些函数进行深度测试，从而提高漏洞发现的针对性。然而，该方法没有深入到函数内部的变量状态层面，特别是对于涉及数组操作的函数，缺乏对数组状态的精细化分析。

除了基于代码结构和函数重要性的引导策略外，目标状态驱动的引导策略也取得了显著进展。Cha[3]等人提出了一种目标状态驱动的定向模糊测试框架 SDFUZZ，该系统定义了明确的目标状态集合，通过计算当前程序状态与目标状态之间的状态距离来引导测试方向。然而，该方法对程序状态的建模相对粗粒度，主要关注程序的控制流状态和整体行为状态，没有对复杂数据结构如数组的状态进行精细化建模。

程序状态的感知和建模是提高模糊测试效果的关键技术之一。在变量状态感知方面，Gan[4]等人提出了关键变量状态感知的定向灰盒模糊测试方法。该方法通过静态分析识别程序中的关键变量并在测试过程中动态跟踪这些变量的状态变化。然而，该方法的的状态模型相对简单，主要关注变量的取值范围和边界条件，没有充分考虑复杂数据结构如数组的多维状态特征。在有状态系统的测试方面，Chen[5]等人专门针对 REST API 设计了有状态模糊测试框架 RESTler。该框架通过维护 API 的状态和会话信息，基于状态转换生成测试用

例。虽然RESTler的状态建模和状态转换的思想对本研究具有重要的借鉴意义，但其状态模型主要关注 API 级别的会话状态和资源状态，而不是程序内部的变量状态，因此不能直接应用于数组状态的建模。在启发式状态探索方面，Yuan[6]等人创造性地将生物进化中的物种发现算法应用于软件测试领域，提出了 STADS 系统。该方法将测试用例视为生物个体，通过聚类分析识别出测试输入中的不同"物种"，为每个物种分配不同的变异和选择策略。然而，该方法对状态的识别主要依赖于测试输入的特征聚类，而不是程序内部的变量状态分析。

尽管已有研究在定向模糊测试和状态感知方面取得了显著进展，但仍然存在一些共同的局限性。首先，大多数现有方法主要关注代码覆盖率问题，而没有充分考虑变量状态的细节信息。代码覆盖率虽然能够反映测试的广度，但在触发与特定变量状态相关的错误方面能力不足。其次，现有的状态感知方法大多将状态视为整体进行建模，缺乏对不同类型变量的差异化处理。特别是在处理数组类型变量时，现有方法通常将数组视为普通的内存区域，没有充分考虑到数组变量的特殊性质，如索引范围、索引分布、元素类型、访问模式、长度变化等。最后，现有的测试用例生成策略大多基于随机变异或确定性变异，缺乏对目标状态的主动引导，缺乏针对数组状态特征的专门变异算子。这些局限性正是本课题要解决的核心问题，也为本研究提供了明确的研究方向和创新空间。

三、初步拟定的执行方案

本课题将基于现有模糊测试工具进行改进和扩展，选择 AFL（American Fuzzy Lop）作为基础平台。AFL 是一款广泛使用的灰箱模糊测试工具，具有覆盖率引导、种子变异、能量调度等成熟机制，为本课题的实现提供了良好的基础。本课题将在 AFL 的基础上，添加数组状态识别、状态多样性度量和基于状态的测试用例生成等功能模块，形成完整的数组状态引导模糊测试工具。执行方案的技术路线共包括四个核心模块的设计与实现。

1. 数组变量状态识别模块。

该模块采用静态分析与动态插桩相结合的方法，静态分析阶段通过编译器前端（如LLVM）识别目标程序中的数组变量声明和定义，收集数组的基本属性信息，包括数组名称、类型、长度、作用域等。动态插桩阶段在编译时对数组访问操作进行插桩，记录每次数组访问的索引值、访问类型（读操作或写操作）、访问上下文（调用栈信息）等运行时状态信息。插桩采用轻量级设计，最小化对程序执行效率的影响，状态信息通过共享内存反馈给模糊测试引擎。

2. 状态多样性度量模块。

该模块负责维护和管理数组变量的状态信息，并计算状态多样性指标。状态信息采用哈希表结构存储，以数组标识符为键，以状态向量为值。状态向量包含数组长度、索引分布统计、边界访问次数、异常访问次数等多个维度。状态多样性计算采用信息熵和覆盖度的组合度量，信息熵反映状态分布的广度，覆盖度反映对特殊状态（如边界状态）的探索深度。模块定期对测试用例集合进行状态多样性评估，识别状态空间中未充分探索的区域，为测试用例生成提供引导信息。

3. 基于状态的测试用例生成模块。

该模块是本课题的核心创新，负责根据状态多样性指标生成定向测试用例。模块设计了一组针对数组状态的专门变异算子，包括索引边界变异算子（逐步增加或减少索引值直至接近或超出边界）、长度调整变异算子（修改输入数据长度以触发缓冲区溢出）、模式识别变异算子（基于历史访问模式生成特定输入）等。变异策略采用两阶段方法，第一阶段根据状态多样性指标选择目标状态和优先级，第二阶段应用相应的变异算子生成测试输入。模块还包括自适应机制，根据执行反馈调整变异参数，如变异强度、变异频率等。

4. 集成与反馈模块。

该模块负责协调上述三个模块的工作流程，并与 AFL 的原有机制进行集成。集成方案采用模块化设计，通过定义清晰的接口与 AFL 的种子选择、变异执行、覆盖率反馈等模块进行交互。反馈机制将数组状态信息与覆盖率信息进行融合，形成综合的测试用例评估指标。在种子选择阶段，优先选择能够增加状态多样性的种子；在变异阶段，根据目标状态选择合适的变异算子；在执行阶段，收集状态信息并更新状态多样性指标；在评估阶段，综合考虑覆盖率增量和状态多样性增量来确定种子的保留或丢弃。

整个执行方案遵循原型开发方法，采用迭代增量式的开发策略。首先实现基础的状态识别和收集功能，验证技术可行性；然后实现状态多样性度量功能，完善状态空间的表示和评估；接着实现定向变异算子，形成完整的测试用例生成能力；最后进行系统集成和性能优化，确保工具的实用性和可靠性。

四、具体进度计划

1 – 2 周：学习背景知识，完成开题报告；
3 – 5 周：需求分析；
6 – 8 周：系统设计；
9 - 10 周：系统实现及测试；
11-12 周：撰写及修改毕业设计报告；
13-14 周：答辩准备及最终答辩。

五、参考文献

[1] Ruixiang Qian, Quanjun Zhang, Chunrong Fang, Ding Yang, Shun Li, Binyu Li, and Zhenyu Chen. DiPri: Distance-Based Seed Prioritization for Greybox Fuzzing [J]. ACM Transactions on Software Engineering and Methodology, 2025, 34(1): Article 26.

[2] Ruixiang Qian, Quanjun Zhang, Chunrong Fang, Lihua Guo, and Zhenyu Chen. FunFuzz: Greybox Fuzzing with Function Significance [J]. ACM Transactions on Software Engineering and Methodology, Accepted.

[3] Marcel Böhme, László Szekeres, and Jonathan Metzman. On the reliability of coverage-based fuzzer benchmarking [C]. In Proceedings of the 44th International Conference on Software Engineering (ICSE '22), May 21-29, 2022, Pittsburgh, PA, USA: 1621-1633.

[4] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. RESTler: stateful REST API fuzzing [C]. In Proceedings of the 41st International Conference on Software Engineering (ICSE '19), May 25-31, 2019, Montreal, Quebec, Canada: 748-758.

[5] Marcel Böhme. STADS: Software Testing as Species Discovery [J]. ACM Transactions on Software Engineering and Methodology, 2018, 27(2): Article 7.

[6] Xu Chen, Ningning Cui, Zhe Pan, Liwei Chen, Gang Shi, Dan Meng. Critical Variable State-Aware Directed Greybox Fuzzing [C]. 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE '25), April 27-May 3, 2025, Ottawa, Canada: 141-152.

[7] Penghui Li, Wei Meng.SDFUZZ: Target States Driven Directed Fuzzing [C]. 2024 33rd USENIX Security Symposium, August 14-16, 2024, Philadelphia, PA, USA: 2441-2457.

指导教师批阅意见	指导教师(签名)：　　年 月 日